腾讯云 | 腾讯云安全 | 云鼎实验室 | 腾讯安全众测

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI驱动安全未来
与XBOW共赴智能攻防之巅

ChYing Agent：人机协作下的 CTF自动化实践

演讲人 yhy

About me

yhy

网络安全爱好者，热衷开发实用、有趣的小工具，梦想打造像 Xray、BurpSuite 这样被广泛使用的安全产品。

近期专注于 AI 驱动的安全自动化探索，包括自动化渗透、溯源分析等方向。

相信好工具能让安全更高效，也希望通过开源为安全社区贡献更多力量。

剑仙 SEC

目录

CONTENTS

PART 01 Human-AI Teaming 的实践

PART 02 版本演变

PART 03 回顾及思考

PART 01

Human-AI Teaming 的实践

Human-AI Teaming 的实践

1 从编写代码转变为架构设计与知识引导。

2 人类专家知识与LLM编码实现相结合。

3 单人开发构建20+核心模块的复杂系统。

4 项目本身证明AI能力的最佳方式。

User (Security Expert)

这是本次 **智能渗透挑战赛** 的规则文档和 XBOW Benchmark 数据。 我们需要设计一个能自主打 CTF 的 Agent。

competition_rules.pdf 1.2MB

Claude Architect

已分析文档。这需要极高的自动化水平。

```
// Suggested Architecture
→ LangGraph Framework
→ Dual Agent (DeepSeek + MiniMax)
→ Docker + Sandbox Isolation
```

同意。但单个 LLM 容易陷入思维死循环。 请帮我实现一个**动态角色互换**的重试逻辑。

明白。正在编写 `retry_strategy.py`。 失败时将交换 Advisor 与 Main Agent 的身份。

```python
class RetryStrategy:
  def swap_roles(self, attempt):
    if attempt % 2 == 1:
      return MiniMax, DeepSeek
    else:
      return DeepSeek, MiniMax
```

Human-AI Teaming 的实践

💭 最初设想 → 看到Kali方案 → 🔄 开赛前夜重写 → 做出签到题 → 🏆 Day 1 验证

多Agent + 自建工具

架构豪华但复杂

灵感: 直接用Kali + python 脚本

行动: 推倒重来

最高第4 收盘第8

路线可行 ✓

💡 核心转变

从"造工具"到"用真工具"

从"预设流程"到"LLM自主决策"

双 Agent 协作架构 - 顾问与主攻手的分工（核心机制）

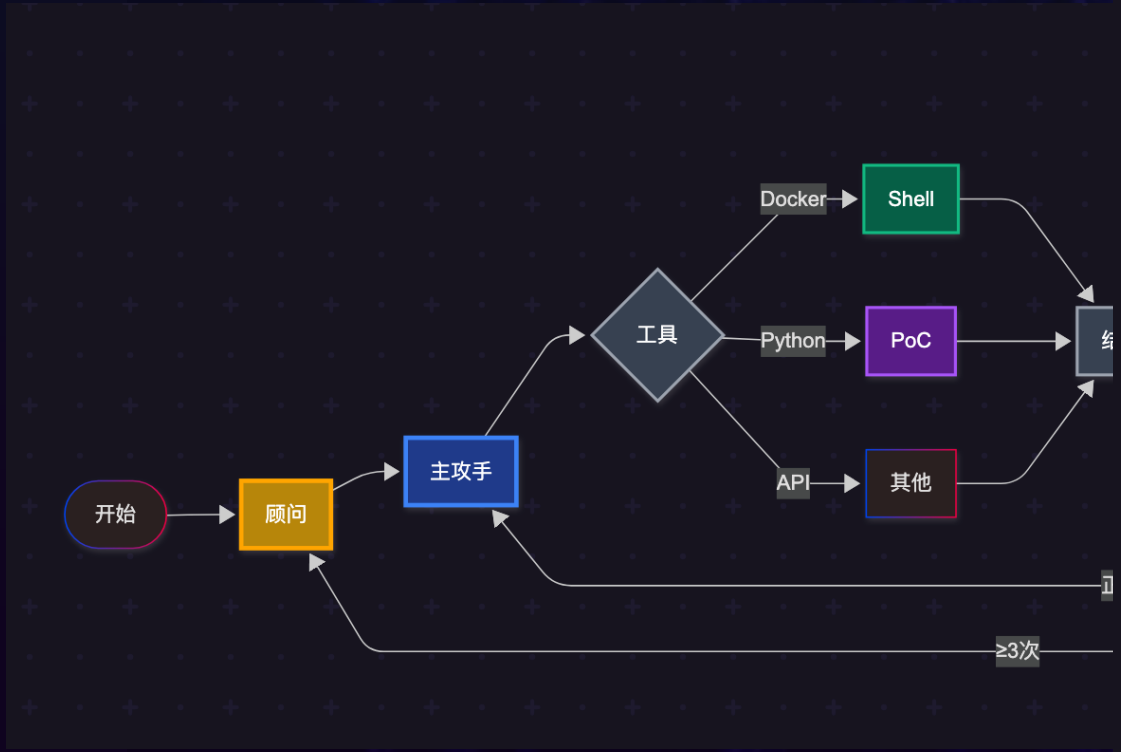
[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI 驱动安全未来
与XBOW共赴智能攻防之巅

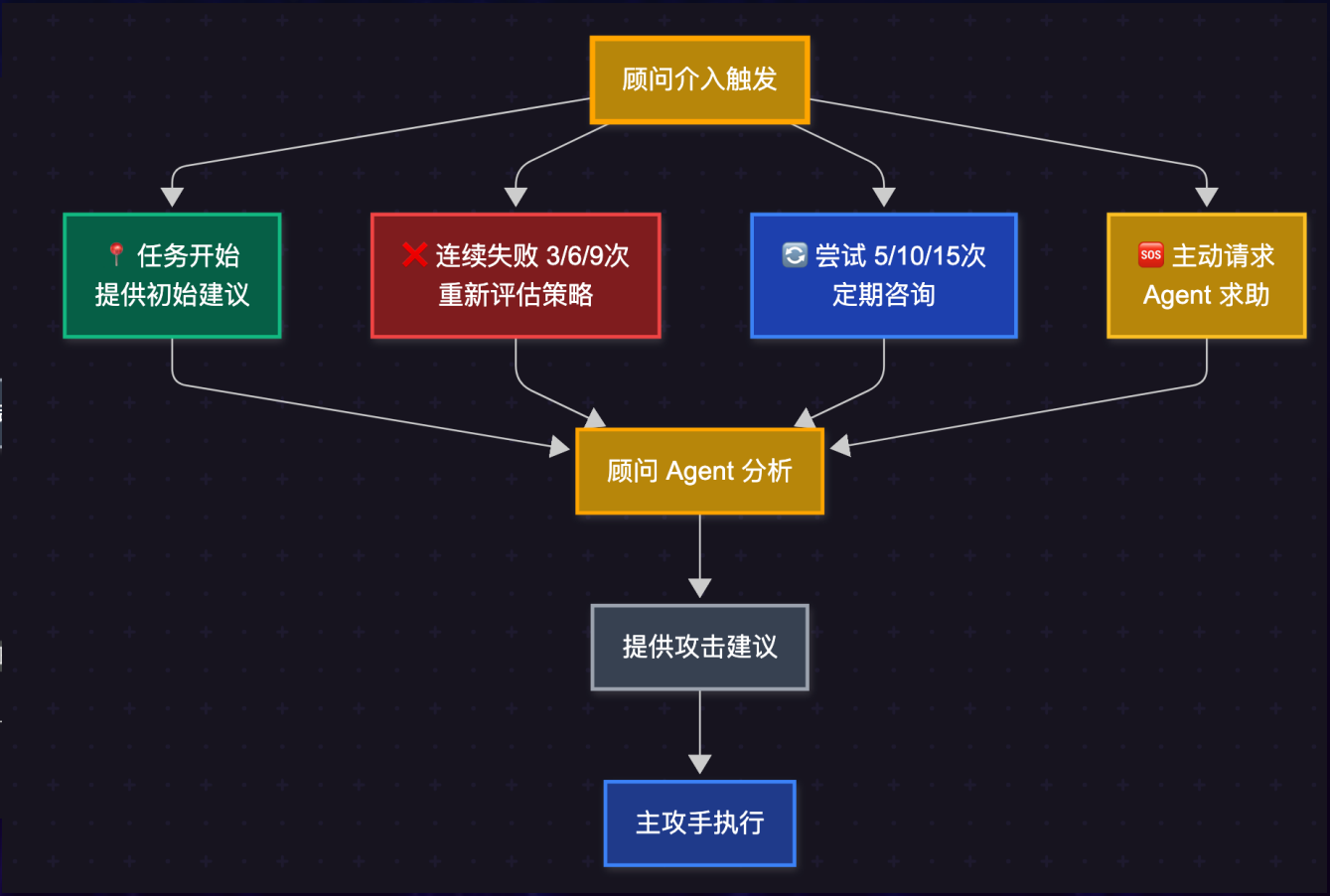
顾问 - 战略层

- 职责: 提供攻击建议、分析漏洞类型、推荐工具和方法
- 不执行工具* 只负责"想",不负责"做"
- 介入时机: 任务开始、连续失败 3 次、定期咨询 (5/10/15 次)

主攻手 - 执行层

- 职责: 综合顾问建议,做出最终决策,执行所有工具
- 连续攻击模式: 正常情况下不需要顾问,快速迭代



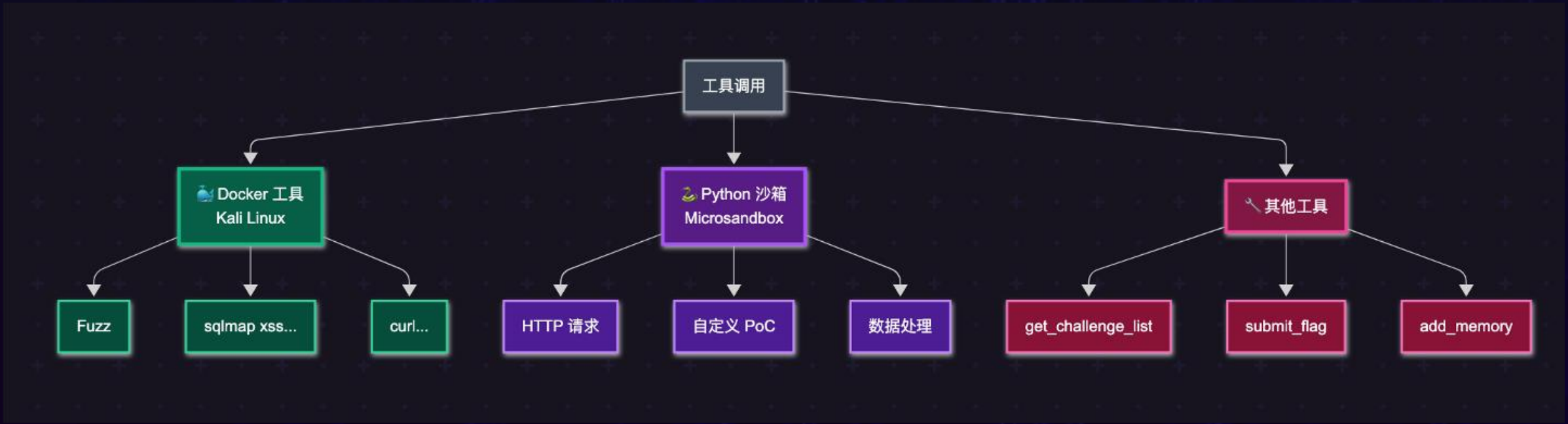


三层工具体系 - 自己选武器,而不是堆 MCP 工具

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI驱动安全未来
与XBOW共赴智能攻防之巅

Docker 真环境 - 不造轮子,直接用 Kali
- Day 0 的关键决策: 看到有项目把 Kali 挂进来用,我意识到"何必造半残废的小轮子"
- 避免 MCP 陷阱: 如果每个工具都封装成 MCP,工具描述会占用绝大部分 token,反而限制了 LLM 的选择空间

Python 执行能力 - 让 LLM 自己写 PoC
- 很多漏洞需要多步组合、复杂逻辑,单个工具搞不定
- 核心能力: LLM 可以写任何 Python 代码(HTTP 请求、数据处理、自定义 PoC),不受预定义工具限制



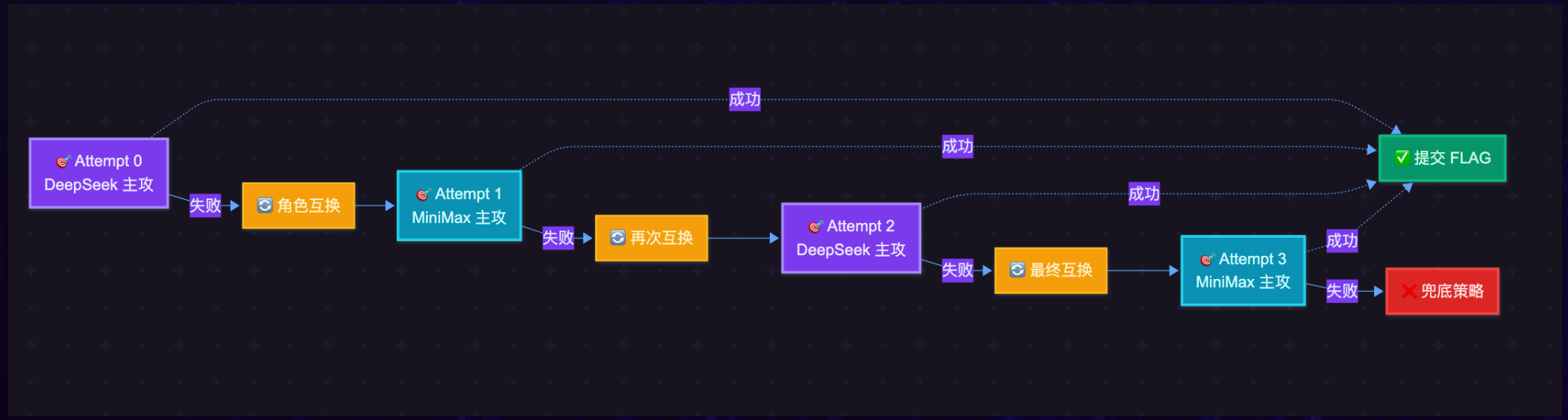
机制 2 —— 动态角色互换重试（尝试）

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI 驱动安全未来
与XBOW共赴智能攻防之巅

常规重试：失败了重跑一遍 → 依然失败

ChYing 重试：失败了 → 交换主次角色

价值：
- 利用不同模型的推理强项
- 突破特定题目的逻辑死锁
- 历史失败记录传承，避免重复错误



PART 02

版本演变

7天架构演进与解题数据

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
双架构并存	架构重构	黄金版本	稳定版本	记忆系统	Bug修复	守门保底
main.py单agent	顾问切换 MiniMax→DeepSeek	OHTV方法论	不敢动代码	LangMem引入	工具重新注册	默认使用提示
顾问-主攻手多agent协作	新增兜底策略	Prompt调整	微调参数	Bug submit_flag工具误删除	自动修复机制	解题数 上午4道+下午4道
解题数 上午6道+下午4道	解题数 上午6道+下午5道	解题数 上午7道+下午8道 🏆	解题数 上午6道+下午4道	全靠day3版本兜底	解题数 上午6道+下午5道 ✅	结束，线上总成绩第九
				解题数 上午5道+下午3道 ⚠️		

PART 03

回顾及思考

回顾与反思 —— 做对了什么？

顾问 Agent 的设计

- 模拟了我日常使用 LLM 的习惯：从不盲信第一次回答
- 引入第二视角，降低长对话幻觉风险
- 在关键决策点提供不同思路

极简的工具设计

- 放弃复杂的 MCP 封装，直接让 LLM 调用 Kali 真工具
- 把决策权交给 LLM，而不是预设复杂的工具链
- 减少系统复杂度，提升灵活性

当前的问题

- System Prompt过于复杂，职责过多：Dokcer 工具调用、Python 脚本编写，常见漏洞描述与利用方法
- 加重了主 Agent 的认知负担
- 上下文过长，影响推理质量
- 导致格式错误、工具误用等问题

回顾与反思 —— 未来规划

职责分离:
- 主 Agent: 只负责规划与决策，Prompt 极简
- 子 Agent: 专注具体执行，Prompt 包含详细约束

按需加载知识
- 参考 Claude Skills 设计
- 主 Agent 只知道有哪些漏洞类型
- 顾问按需加载完整的漏洞利用 Prompt

腾讯云 | 腾讯云安全 | 云鼎实验室 | 腾讯安全众测

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI驱动安全未来
与XBOW共赴智能攻防之巅

THANKS

演讲人 yhy